

第一章-python的基本概念

Python , 解释器, IDE

- Python 是一门**解释执行**的高级语言。源代码由解释器读取并运行。
- 解释器是用于运行python代码的软件
- IDE是用于编辑python代码的软件，通常可以在内部自动调用解释器完成运行

解释执行与编译执行

编译执行：

1. **编译阶段**：编译器一次性将源代码（如 C、C++）翻译成**机器码**（目标平台的二进制指令）。
2. **运行阶段**：CPU 直接执行机器码文件（可执行文件）。

解释执行

1. **解释器逐行读取**源代码。
2. **即时分析并执行**：将代码翻译成中间形式或直接调度相应操作。

PIP 模块

`pip` 是 Python 官方推荐的**包管理工具**（Python Installer for Packages），用于**安装、升级、卸载**第三方库。

第二章 python的基础语法

变量

- 变量：在 Python 中，变量是对对象的引用**（名称 → 对象的映射）。
- 赋值：使用 `=` 将对象绑定到变量名。

```
1 x = 10          # 整数
2 name = "Ada"    # 字符串
```

- 命名规则
 - 必须以字母或下划线 `_` 开头，后续可包含字母、数字、下划线。
 - 区分大小写：`age` 与 `Age` 是不同变量。
 - 不能与保留关键字相同（可用 `import keyword; keyword.kwlist` 查看）。
 - 推荐使用 **小写 + 下划线**（snake_case）命名变量。
- 常见类型
 - **数值类型**：`int`（整数）、`float`（浮点数）、`complex`（复数）
 - **布尔类型**：`bool`（`True` / `False`）
 - **字符串类型**：`str`
 - **容器类型**（后续章节详述）：`list`、`tuple`、`dict`、`set`
 - python对变量的类型没有c++一样严格要求，变量只是对象的引用，因此在编码过程中需要注意变量类型的变化。

- **类型转换 (type casting)** 是将一个数值从一种数据类型转换为另一种数据类型的过程。
- **主动使用内置函数进行转换：**

目标类型	转换函数	示例	结果
整数	<code>int(x)</code>	<code>int(3.7)</code>	3 (向零取整)
浮点数	<code>float(x)</code>	<code>float(3)</code>	3.0
复数	<code>complex(x)</code>	<code>complex(3)</code>	(3+0j)
布尔	<code>bool(x)</code>	<code>bool(0)</code>	False

```

1 x = 3.9
2 print(int(x))      # 3
3 print(float(3))    # 3.0
4 print(complex(2))  # (2+0j)
5 print(bool(5))     # True

```

- **被动 (隐式) 类型转换**

当运算中出现不同类型的数值时，Python 会自动将精度低的类型提升为精度高的类型，以避免数据丢失。

转换规则 (简化版)：

- `int + float` → `float`
- `int + complex` → `complex`
- `float + complex` → `complex`

```

1 result = 3 + 2.5      # 结果是 5.5 (float)
2 result = 3 + 4j       # 结果是 (3+4j) (complex)

```

- 注意： `int()` 对浮点数向零取整

```

1 int(3.9)  # 3
2 int(-3.9) # -3

```

- `float("3.14")` 与 `int("3.14")` 区别

```

1 float("3.14") # 3.14
2 int("3.14")   # ValueError (不能直接把带小数点的字符串转为 int)

```

基本的输入输出

从键盘获取输入

- 使用内置函数 `input([prompt])` 从终端读取用户输入，返回**字符串类型**。
- 读取到的字符串末尾的换行符会被自动去掉,但是不会去除：前后空格、制表符 `\t`、换行前的空格等等

```

1 name = input("请输入你的名字：")

```

- 注意： `input()` 的返回值总是字符串，需要时需显式转换类型：

```
1 age = int(input("请输入年龄："))
```

`split()` 是 **字符串类型** (`str`) 的方法，用于**按照指定分隔符将字符串拆分为列表**

```
1 str.split(sep=None, maxsplit=-1)
```

`sep` (可选)

- 分隔符字符串。
- `None` (默认)：会以**任意连续的空白字符** (空格、制表符、换行符等) 作为分隔符，并自动去掉首尾空白。

`maxsplit` (可选)

- 最大分割次数，默认 `-1` 表示不限制。
- 如果指定为 `n`，最多分割 `n` 次，得到的列表长度最多为 `n+1`

输出到terminal中

使用 `print(objects, sep=' ', end='\n')` 将对象转为字符串并输出到终端。

参数：

- `sep`：多个对象之间的分隔符 (默认空格) 。
- `end`：输出末尾的结束符 (默认换行符 `\n`) 。

```
1 print("A", "B", "C")           # 默认空格分隔
2 print("A", "B", sep="-")       # 自定义分隔符
3 print("Hello", end="!")        # 不换行
```

基本的数学运算

数学运算符

运算符	名称 / 功能	示例 (a=7 , b=3)	结果 / 说明
<code>+</code>	加法	<code>a + b</code>	10
<code>-</code>	减法	<code>a - b</code>	4
<code>*</code>	乘法	<code>a * b</code>	21
<code>/</code>	浮点除法	<code>a / b</code>	2.3333... (float)
<code>//</code>	地板除 (整除)	<code>a // b</code>	2 (向下取整)
<code>%</code>	取余 (模运算)	<code>a % b</code>	1 (余数符号与被除数相同)
<code>**</code>	幂运算 (乘方)	<code>a ** b</code>	343 (7^3)
<code>-x</code>	负号 (取相反数)	<code>-a</code>	-7
<code>+x</code>	正号 (数值不变)	<code>+a</code>	7
<code>abs(x)</code>	绝对值	<code>abs(-7)</code>	7
<code>divmod(a,b)</code>	商与余数	<code>divmod(7,3)</code>	(2 , 1)

赋值运算符

运算符	名称 / 功能	示例 (初始 a=7 , b=3)	结果 (a 的值)	等价于
<code>=</code>	赋值	<code>a = b</code>	3	—
<code>+=</code>	加并赋值	<code>a += b</code>	10	<code>a = a + b</code>
<code>-=</code>	减并赋值	<code>a -= b</code>	4	<code>a = a - b</code>
<code>*=</code>	乘并赋值	<code>a *= b</code>	21	<code>a = a * b</code>
<code>/=</code>	除并赋值 (浮点)	<code>a /= b</code>	2.3333...	<code>a = a / b</code>
<code>//=</code>	地板除并赋值	<code>a //= b</code>	2	<code>a = a // b</code>
<code>%=</code>	取余并赋值	<code>a %= b</code>	1	<code>a = a % b</code>
<code>**=</code>	幂运算并赋值	<code>a **= b</code>	343	<code>a = a ** b</code>
<code>&=</code>	按位与并赋值	<code>a &= b</code>	按位结果	<code>a = a & b</code>
<code> =</code>	按位或并赋值	<code>`a</code>	<code>= b`</code>	按位结果
<code>^=</code>	按位异或并赋值	<code>a ^= b</code>	按位结果	<code>a = a ^ b</code>
<code><<=</code>	按位左移并赋值	<code>a <<= 1</code>	左移 1 位的结果	<code>a = a << 1</code>
<code>>>=</code>	按位右移并赋值	<code>a >>= 1</code>	右移 1 位的结果	<code>a = a >> 1</code>

比较运算符

运算符	名称 / 功能	示例 (a=7, b=3)	结果 / 说明
<code>==</code>	等于	<code>a == b</code>	<code>False</code>
<code>!=</code>	不等于	<code>a != b</code>	<code>True</code>
<code>></code>	大于	<code>a > b</code>	<code>True</code>
<code><</code>	小于	<code>a < b</code>	<code>False</code>
<code>>=</code>	大于等于	<code>a >= b</code>	<code>True</code>
<code><=</code>	小于等于	<code>a <= b</code>	<code>False</code>
<code>is</code>	对象标识是否相同	<code>a is b</code>	比较内存地址 (通常用于比较对象是否同一引用)
<code>is not</code>	对象标识是否不同	<code>a is not b</code>	与 <code>is</code> 相反
<code>in</code>	成员运算	<code>3 in [1,2,3]</code>	<code>True</code> (是否在可迭代对象中)
<code>not in</code>	非成员运算	<code>4 not in [1,2,3]</code>	<code>True</code>

- 运算优先级 (高 → 低)

```
1 | ** > 一元运算符 +x、-x > *、/、//、% > +、-
```

import , 包, 模块 (Module)

- 模块是一个 **Python 文件** (以 `.py` 结尾) , 包含变量、函数、类等定义, 可以被其他 Python 程序导入使用。如: `math.py`、`os.py` 等标准库文件。
- 包是一个包含 **多个模块** 的 **目录**, 目录中必须有一个 `__init__.py` 文件 (Python 3.3+ 中即使省略也可作为包, 但加上更明确) 。
 - **作用**: 组织和管理模块, 形成层次结构, 避免命名冲突。

```
1 | my_package/ # 包
2 |   __init__.py #定义包的基本结构或一些初始化步骤
3 |   geometry.py #模块
4 |   algebra.py  #模块
```

- `import` 用于导入包或者模块

导入模块:

```
1 | import math          # 导入整个模块
2 | import math as m     # 导入并取别名
3 | from math import pi  # 从模块导入特定对象
4 | from math import sqrt as s # 导入并重命名
```

导入包:

```
1 | import my_package
```

作用：执行 `my_package/__init__.py` 中的代码。

如果 `__init__.py` 中定义了变量、函数、类，就可以通过 `my_package.对象名` 使用。

如果 `__init__.py` 为空，仅表示这是一个包，不能直接访问内部模块的内容。

从包中导入模块：

```
1 | import my_package.module_a
2 | from my_package.module_a import func
```

导入子包：

```
1 | import my_package.sub_package.module_c
2 | from my_package.sub_package import module_c
3 | from my_package.sub_package.module_c import func_c
```

- python搜索包的机制：优先查找当前工作目录，不存在则寻找 `sys.path`